CS1200: Intro. to Algorithms and their Limitations Profs. Anshu and Nesson

Problem Set 6

Harvard SEAS - Fall 2026 *Due: Thu Nov. 12, 2026 (11:59pm)*

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.' A problem set on the border between two letter grades cannot be rounded up if collaborators are not noted.

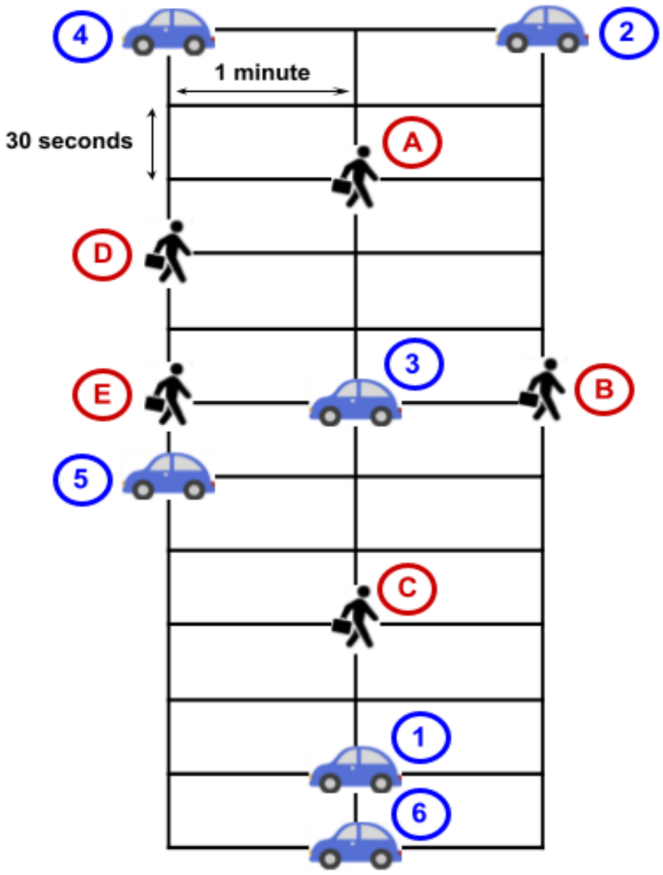
Your name:
Collaborators and External Resources:
No. of late days used on previous psets:
No. of late days used after including this pset:

The purpose of this problem set is to practice modeling problems using graphs, reinforce understanding of the matching algorithm we learned, practice the resolution algorithm and reductions to SAT, and think about ethical issues raised when modeling real-world problems for algorithmic solution.

1. (Matching Algorithms) Another practical application of matching algorithms is planning logistics, like in the following example from (fictional) ridesharing service Lyber in (real) New York City's Times Square. When a customer books a Lyber ride, the ride request is sent to a Lyber server and combined with others to create a schematic like the one drawn in the map below.

   Given a schematic like this, Lyber's goal is to serve as many customers (labeled A–E in the map) as possible, by assigning each one to a driver (labeled 1–6 in the map). For simplicity, each customer and driver is at an intersection, and assume driving between adjacent streets (vertical segment) takes 30 seconds, and driving between adjacent avenues (horizontal segments) takes 1 minute. However, the one twist is that they want to make sure that *no customer is waiting for longer than 2 minutes*. They also do not want to assign a driver to more than one customer at once, since serving a single customer can take more than 2 minutes.

   (a) To perform the assignment, they reduce to Maximum Matching in bipartite graphs. Draw a bipartite graph corresponding to the drivers and customers in the map above.



(b) The Lyber app first prioritizes customers on Broadway, so they initially assign customer A to driver 3 and customer C to driver 5. Using the algorithm from class, find a *maximum matching* in the bipartite matching graph you've drawn, starting from the initial matching of A to 3 and C to 5. Draw pictures showing the sequence of matchings and augmenting paths you find. (No need to break down the steps of the algorithm to find the augmenting paths.)

2. (Resolution) Use the algorithm `ResolutionInOrder` (Algorithm 18.2 from Section 18.1 in the textbook) to decide the satisfiability of the following formula, and if it is satisfiable, use the algorithm `ExtractAssignment` (Algorithm 18.4 in Section 18.2 of the textbook) to obtain a satisfying assignment. (Please make sure to follow both algorithms *exactly*, including the order in which the clauses are processed. A correct final solution that does not show all of the intermediate steps of the algorithms will not receive full score. You may omit resolvents that are 1 or duplicates of existing clauses.)

$$\varphi(x_0, x_1, x_2, x_3) = (x_0 \vee x_1) \wedge (\neg x_0 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

3. (Reductions to SAT) In Section 15.3 of the textbook, we saw an efficient algorithm to solve the Maximum Matching problem in Bipartite Graphs. A variant of Maximum Matching

is Maximum 3-D Matching, where we are given not a graph $G = (V, E)$ but a "3-uniform hypergraph" $H = (V, E)$, where the *hyperedges* $e \in E$ now are *triples* of vertices. Analogously to restricting to bipartite graphs, we restrict to *tripartite* hypergraphs where V is the union of 3 disjoint sets $V_0 \cup V_1 \cup V_2$ and every hyperedge $e \in E$ has exactly one vertex from each of V_0, V_1, V_2. For example, hyperedges may consist of compatible patient-donor-timeslot triples (if there are only certain timeslots in which a patient and donor are available for a surgery), or compatible surfer-surfboard-fins triples (if each surfer only likes to ride certain surfboards with certain fins installed). Now the goal is to find a maximum-sized set $M \subseteq E$ such that every vertex $v \in V$ is contained in at most one edge of M. Unfortunately, unlike matching in graphs, there is no polynomial-time algorithm known for Maximum 3-D Matching, even if we restrict to tripartite hypergraphs and even if we only want to find *complete matchings* — ones that match every vertex in V_0 (i.e. we are looking for a matching of size $|M| = |V_0|$):

Input: Three disjoint finite sets of vertices V_0, V_1, V_2 and a set E of hyperedges such that each $e \in E$ contains exactly one vertex from each of V_0, V_1, and V_2.

Output: A set M of hyperedges such that every vertex is in at most one hyperedge in M, and every vertex in V_0 is in some hyperedge in M (if one exists).

Computational Problem 3dCompleteMatching()

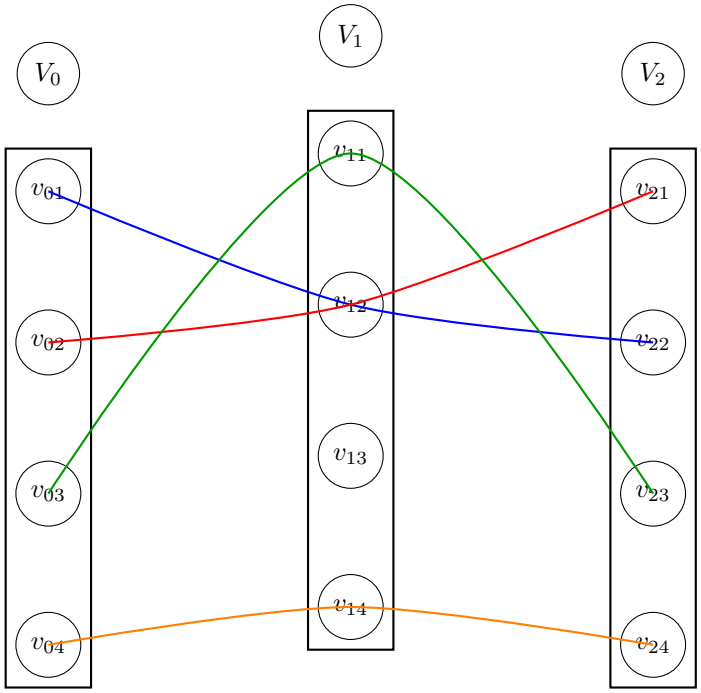


Figure 1: Example of hypergraph with sets of vertices $V_0 = [v_{01}, v_{02}, v_{03}, v_{04}], V_1 = [v_{11}, v_{12}, v_{13}, v_{14}], V_2 = [v_{21}, v_{22}, v_{23}, v_{24}]$ and hyperedges $E = \{(v_{01}, v_{12}, v_{22}), (v_{02}, v_{12}, v_{21}), (v_{03}, v_{11}, v_{23}), (v_{04}, v_{14}, v_{24})\}$

Give a polynomial-time reduction of 3dCompleteMatching to SAT that, on hypergraphs with $n = |V_0| + |V_1| + |V_2|$ vertices and $m = |E|$ hyperedges, makes one oracle query on a formula

with m variables and $O(n + m^2)$ clauses. Clearly describe your preprocessing (what the variables represent and what clauses you construct), the oracle call, and your postprocessing (including what you return if the oracle returns $\perp$).

You do not need to prove the correctness of your reduction or analyze its runtime. *Please keep your response within 12 sentences.*

Thus, even though the fastest known algorithms for 3dCompleteMatching run in exponential time, we can use SAT Solvers to solve it much more efficiently on many instances that arise in practice.

4. (EthiCS reflection) Suppose you are given a bipartite graph $G = (V, E)$ representing possible kidney donor–recipient matches, where the edges represent perfect compatibility matches, with far fewer donors than recipients, and for which there are multiple MAXIMUM MATCHING solutions. No living donors in the graph face any health risks from the procedure. Finally, you have access to the following data for all of the potential recipients: (1) expected life years gained from transplant, (2) expected QALYs gained from transplant, (3) age, (4) past QALYs lived to date, and (5) household income. You are tasked with developing a metric `value` that takes some (or all) of that data and a potential matching $M \subseteq E$ and outputs a score $\texttt{value}(M) \in \mathbb{R}$. Then your engineers will design an algorithm that finds a matching maximizing $\texttt{value}(M)$.

   Describe some core characteristics your metric `value` ought to have and explain why you think those characteristics are ethically justified, drawing from at least one concept or discourse from the ethical & social considerations lecture/chapter as well as your personal judgment.

   Note: As with the previous psets, you may include your answer in your PDF submission, but the answer should ultimately go into a separate Gradescope submission form.

   Good responses are usually around 10 sentences

5. Once you're done with this problem set, please fill out this survey so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses!